

Duy Hoang

CSIT 313

Assignment 5

1a.) When that initial else vanishes, what's left links up with the second if inside. Java pairs each else with the closest free if above it. The example proves the block under else follows the inner condition, never touching the outer one.

1b.) Putting braces around the inner if stops the else from linking to it. That way, the else connects only to the outer if. Braces change how pieces fit together. Now the structure makes clear which if owns the else.

2a.) Right away, the output proves something clear - only firstCondition gets checked. Because of how Java handles logic, it skips what comes next when false appears early. That happens here thanks to the double ampersands linking conditions together. The moment one part fails, everything else drops. You see just a single print statement appear on screen. Nothing triggers past that point. So yes, short-circuit behavior runs exactly as expected.

2b.) Java allows short-circuit evaluation by default, yet here the & symbol ensures both sides run regardless. Even when the initial check fails, execution continues without skipping ahead. You see both messages - firstCondition evaluated plus secondCondition evaluated appear each time. That outcome happens only because & ignores early exits. Full traversal stays guaranteed, unlike with typical logical operators.

3.)When the first version spots a nonzero entry, it jumps straight to the next line with goto.

Instead of that jump, my version sets up a flag called allZero at the start. Each time around, if the number isn't zero, the flag flips to false. Once flipped, a break ends the scanning right away.

Only when nothing triggers that change does the system report which row was fully empty. What makes this code back up my point is how it keeps the original idea yet organizes steps clearly.

Similar in many ways, Java alongside C share features like loops, true-false flags, and exit triggers. Syntax sets them apart - printing in Java looks like `System.out.println` compared to `Console.WriteLine` in C. When accessing grids, Java goes for `x[i][j]`, whereas C picks `x[i, j]`.

Readability runs close, though newcomers might find Java a touch more approachable.